

Greedy



Lecture Flow

- Pre-requisites
- Motivating Problem
- Greedy And Terms
- Properties of Greedy
- Exercises
- Proving Correctness
- Pitfalls
- Quote of the day



Prerequisites

- Arrays



Motivating Problem

The Coin Collector's Dilemma

Problem Statement

You are a cashier at a busy store, and customers are paying with large bills. Your job? Give change using the fewest number of coins possible!

Your Challenge

- You have N dollars that you need to make using a given set of M coin denominations.
- You can use unlimited coins of each type.
- Goal: Use the fewest coins to make exactly N .

Example: Imagine you need to give 87 dollars using coins of 25, 10, 5, and 1.

Feasible vs. Optimal Solution

Before solving our Coin Collector's Dilemma, let's understand an important concept:

Feasible Solution:

A solution that meets the problem's requirements but **isn't necessarily the best**.

Optimal Solution:

A solution that is **both feasible and the best possible** according to a specific criteria.

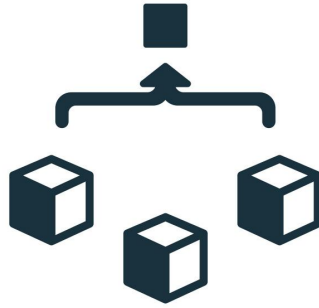
Example in Our Problem

Imagine you need to make 87 dollars using coins {25, 10, 5, 1}.

Feasible Solution 1: Use 87×1 -dollar coins $\rightarrow$  Valid, but not minimal.

Feasible Solution 2: Use $8 \times 10 + 7 \times 1$ $\rightarrow$  Works, but not minimal.

Optimal Solution: {25, 25, 25, 10, 1, 1} $\rightarrow$ Only 6 coins!



GENERALIZATION

Problems like **The Coin Collector's Dilemma** which require finding the **optimal solution** from a set of feasible options belong to a broader class known as **optimization problems**

Optimization Problem

An optimization problem is a type of problem where the goal is to find the **best possible solution** from a set of feasible solutions.

You typically aim to either **maximize** or **minimize** a certain task.

Common components of an optimization problem

1. Objective Function
2. Decision Variables
3. Constraints

1. Objective Function

The **function** that needs to be either maximized or minimized.

Purpose:

Represents the goal or outcome that you want to achieve.

2. Decision Variables

The **variables** that you can adjust or control to influence the objective function.

Purpose:

Represents the parameters or choices that you need to make.

3. Constraints

Restrictions or **limitations** on the values that the decision variables can take.

Purpose:

Ensures that solutions are feasible and realistic.

Can you think of any real life optimization problems?



Real life Optimization Problems

1. Grocery Shopping

Problem: You want to buy groceries with a limited budget and maximize the number of items you can purchase.

Objective Function: Maximize the number of grocery items.

Decision Variables: Quantities of different items to buy.

Constraints: Budget constraint.

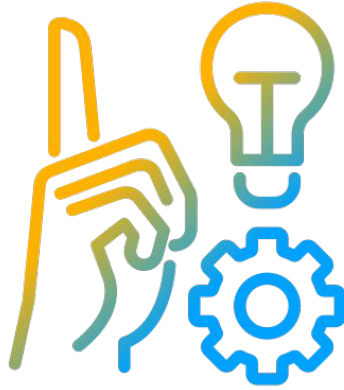
What are the components of the motivating problem?

Problem: Given a total amount N and a set of M coin denominations, find the minimum number of coins needed to make exactly N .

Objective Function: Minimize the total number of coins used.

Decision Variables: The amount of coin used at a time.

Constraints: The total sum of the chosen coins must equal N .



Optimization problems often require **efficient** strategies to find the **best** solution. One common approach is the **greedy method**

How do you think the greedy method solves an optimization problem just by it's name?



Greedy

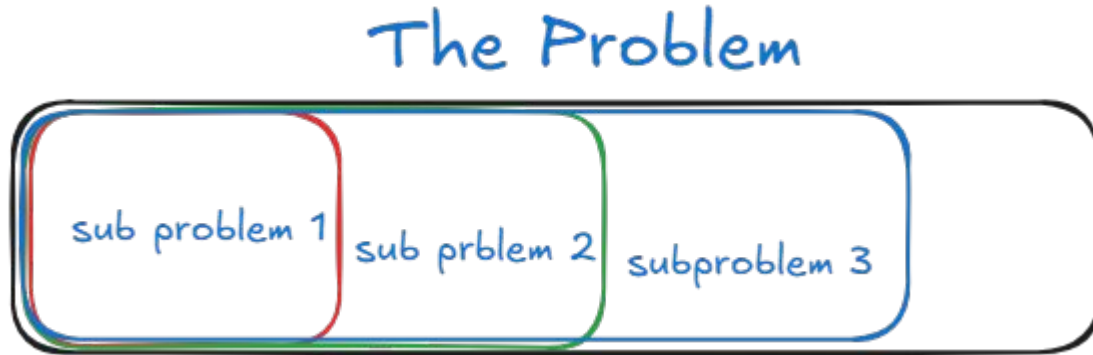
The **greedy method** is an approach to solving problems by making **locally optimal choices** at each stage with the hope of finding a **global optimum**.

It follows the idea that, at each step, the approach selects the **best available option** without considering the consequences of that choice in future steps.

Terms

Subproblem

A subproblem is a **smaller** instance of a larger problem that shares some form of **similarity**.



Subproblem - Example

A2SV is organizing a **coding contest** between three universities: **AAU**, **AASTU**, and **ASTU**. Each university must form a team of **exactly 3 students** to represent them in the competition. Each university is divided into **groups (G61, G62, etc...)**. Each group has scores and within-group ranks for its students based on past performance.

Form the strongest possible team for each university.

Subproblem - Example

A2SV is organizing a **coding contest** between three universities: **AAU**, **AASTU**, and **ASTU**. Each university must form a team of **exactly 3 students** to represent them in the competition. Each university is divided into **groups (G61, G62, etc...)**. Each group has scores and within-group ranks for its students based on past performance.

Form the strongest possible team for each university.

For each university, the **subproblem** is:

👉 **Selecting the Top 3 Performers in Each Group**

Combining the results:

- **Selecting the Best 3 Students for the University**
 - Gather all top students from every group.
 - Select the top 3 among them to form the university's final team.

Local Optimum

A local optimum is a choice that is best among a **specific set of options**.

Global Optimum

A global optimum, also known as the global minimum or maximum, refers to the best possible solution of an optimization problem across the **entire feasible solution space**.

Properties of Greedy

1. **Greedy Choice Property:** A greedy algorithm makes the locally optimal choice at each step without considering the overall structure of the problem.
2. **Optimal Substructure:** The problem should have the property that an optimal solution to the entire problem can be constructed from optimal solutions to its **subproblems**.

Selection Rule

In **greedy**, the **selection rule** is a key decision-making principle that guides the algorithm to choose the **next best option at each step**.

Heuristic

A **heuristic** can be considered a type of **selection rule**, particularly in the context of **optimization algorithms**.

It uses **intuitive shortcuts** and **practical strategies** to quickly find a good enough solution. It favors efficiency over exhaustive optimization.

It is a practical and often simplified rule or strategy that guides decision-making.

Greedy Example With A Heuristic

Imagine you are in [Addis Ababa](#), and you need to drive to one of these five cities for vacation: [Adama](#), [Bahir Dar](#), [Hawassa](#), [Debre Zeyit](#), and [Gondar](#).

The goal is to [minimize travel time](#), but since we do not have exact travel times, we must rely on [heuristic functions](#) to estimate the best route.

Greedy Example With A Heuristic

Imagine you are in **Addis Ababa**, and you need to drive to one of these five cities for vacation: **Adama**, **Bahir Dar**, **Hawassa**, **Debre Zeyit**, and **Gondar**.

The goal is to **minimize travel time**, but since we do not have exact travel times, we must rely on **heuristic functions** to estimate the best route.

1. Straight-Line Distance (Euclidean Distance)
2. Manhattan Distance, $d = |x_2 - x_1| + |y_2 - y_1|$
3. [if available] Road Distance (Actual Distance via Roads)

What type of heuristic is appropriate for the motivating problem?



What type of heuristic is appropriate for the motivating problem?

Heuristic: Always select the **largest** possible coin that does not exceed the remaining amount.

Be careful what you are greedy on

Problem:

Given a set of coin denominations (e.g. 1, 3, 4 cents) and a target amount of 6 cents, find the minimum number of coins required to make the target amount.

Be careful what you are greedy on

Problem:

Given a set of coin denominations (e.g. 1, 3, 4 cents) and a target amount of 6 cents, find the minimum number of coins required to make the target amount.

This fails with greedy!

Greedy Solution : $\{4, 1, 1\}$ $\rightarrow$  Works, but not minimal.

Optimal Solution: $\{3, 3\}$ $\rightarrow$ It was possible to use only 2 coins!

Termination Condition

- A condition that stops a greedy algorithm once the best possible choice is made.
- Prevents unnecessary computations by stopping early when further decisions won't improve the result.

Example – Termination Condition

Problem:

Given a number n , determine if it is prime.

Solution:

- Checking all numbers up to n , if they are the divisors of n .

Example – Termination Condition

Problem:

Given a number n , determine if it is prime.

Solution:

- Checking all numbers up to n , if they are the divisors of n .
- Instead of checking all numbers up to n , we can stop at $\sqrt{n}$.
- Why? If n has a divisor greater than $\sqrt{n}$, it must also have one smaller.

Termination Condition: Stop checking once $i > \sqrt{n}$

Practice Time

Largest Perimeter Triangle

Exercise

1. Jump Game
2. Two City Scheduling



Once we know the right greedy choice, it's easy to implement it.
But how can we know if a greedy choice is right?

Proving Correctness

Proofing or analyzing the correctness and optimality of greedy algorithms involves demonstrating that the algorithm **consistently makes locally optimal choices** at each step, leading to a **globally optimal solution**.

Proofing Mechanisms

1. **Proof by Induction**
2. **Proof by Contradiction**
3. **Exchange Argument**

Proof by Induction

Proof by Induction

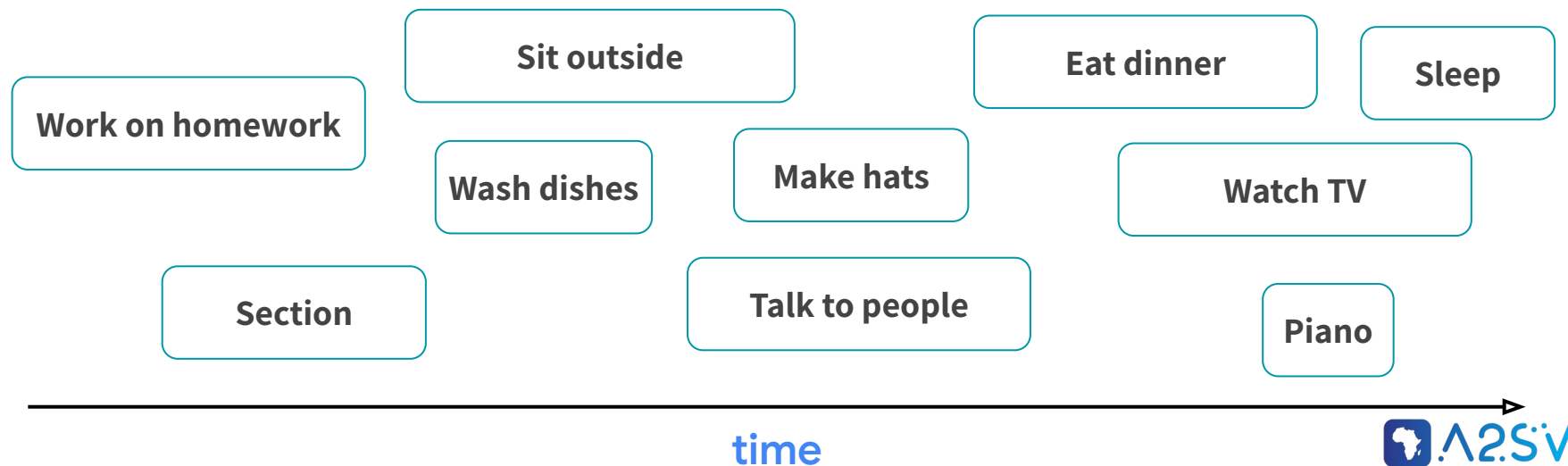
The technique involves proving a statement for a base case and then demonstrating that if the statement holds for any given case, it must also hold for the next case.

ACTIVITY SELECTION PROBLEM

Problem: n activities with start times and finish times

Constraint: All activities are equally important, but you can only do 1 activity at a time!

Objective function: Maximize the number of activities you can do

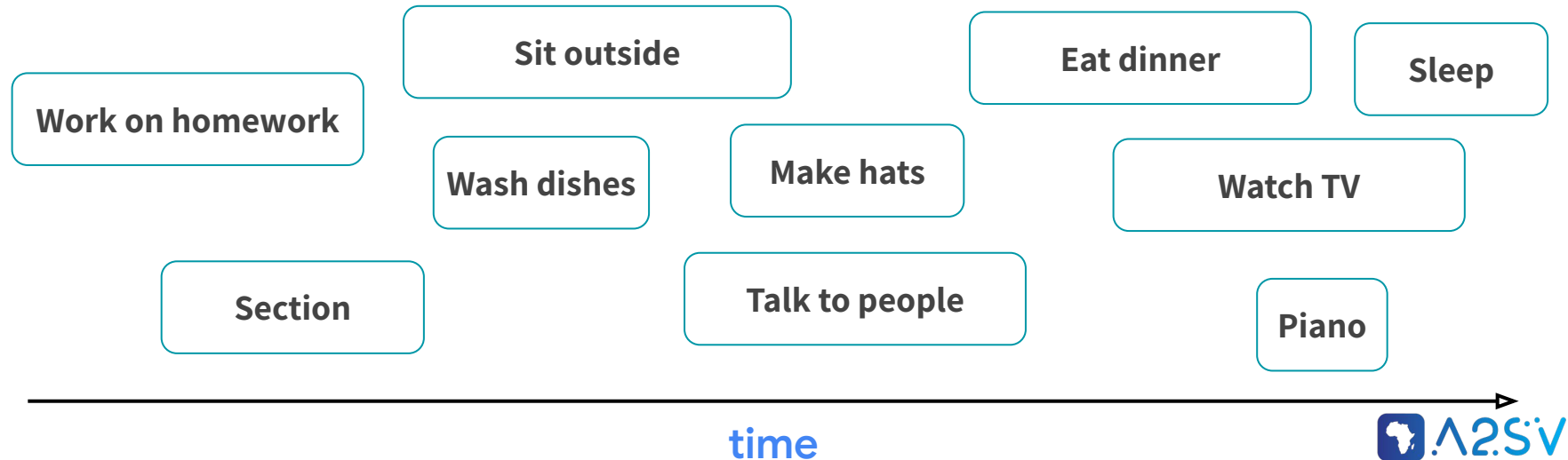


How would you solve this question?



Activity Selection : Options

In what order should you greedily add activities? Here are 4 ideas:



Activity Selection : Options

Be impulsive: choose activities in ascending order of start times

Avoid commitment: choose activities in ascending order of length

Avoid conflicts: choose activities in ascending order of the amount of conflicts

Finish fast: choose activities in ascending order of end times

Activity Selection : Options



counterexample for earliest start time



counterexample for shortest interval



counterexample for fewest conflicts

Activity Selection : Options

1) Be impulsive

2) Avoid commitment

3) Avoid Conflicts

4) Finish fast

Work on hom

Only the fourth option seems to work (this is just our intuition right now, but we need to prove this)!

Sleep

Section

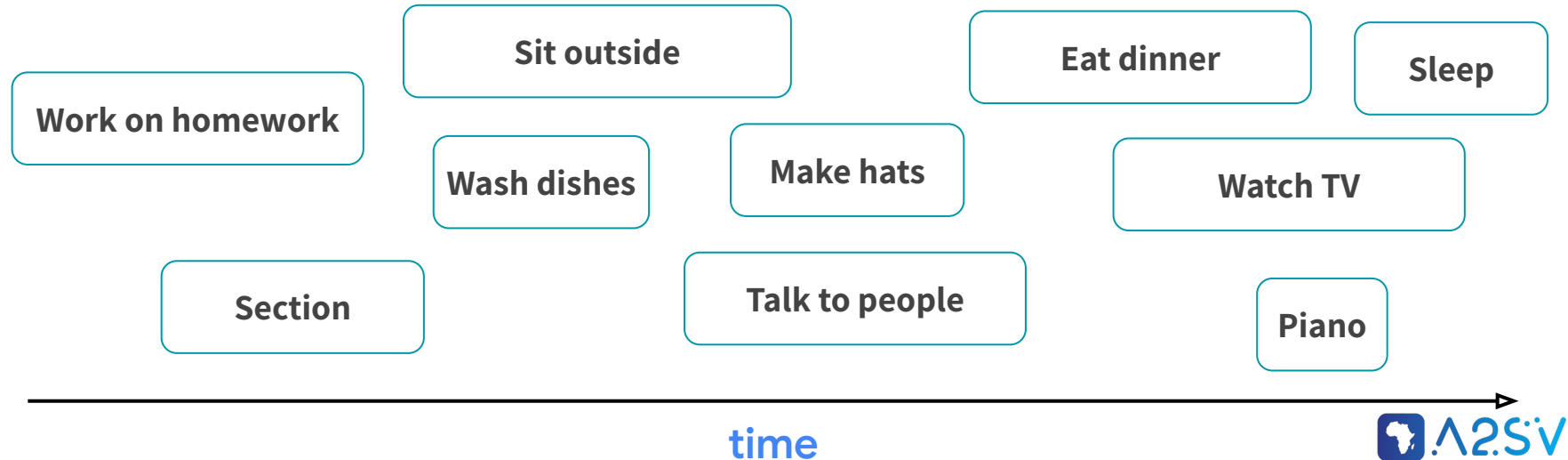
Talk to people

Piano

time

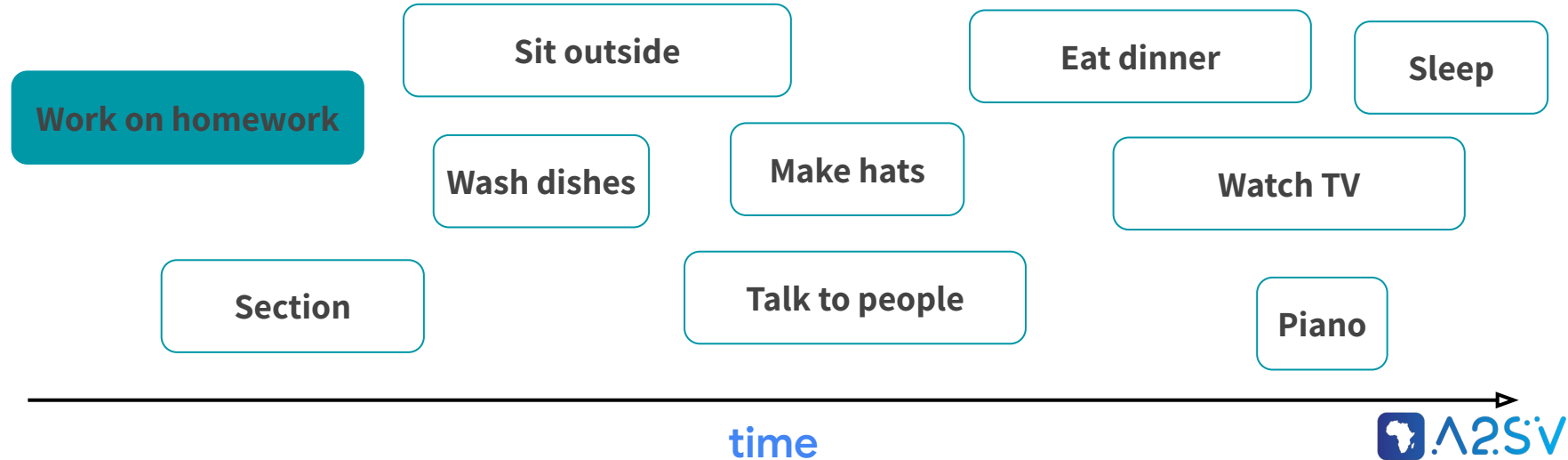
Our Greedy Algorithm

Pick an available activity with the smallest finish time & repeat.



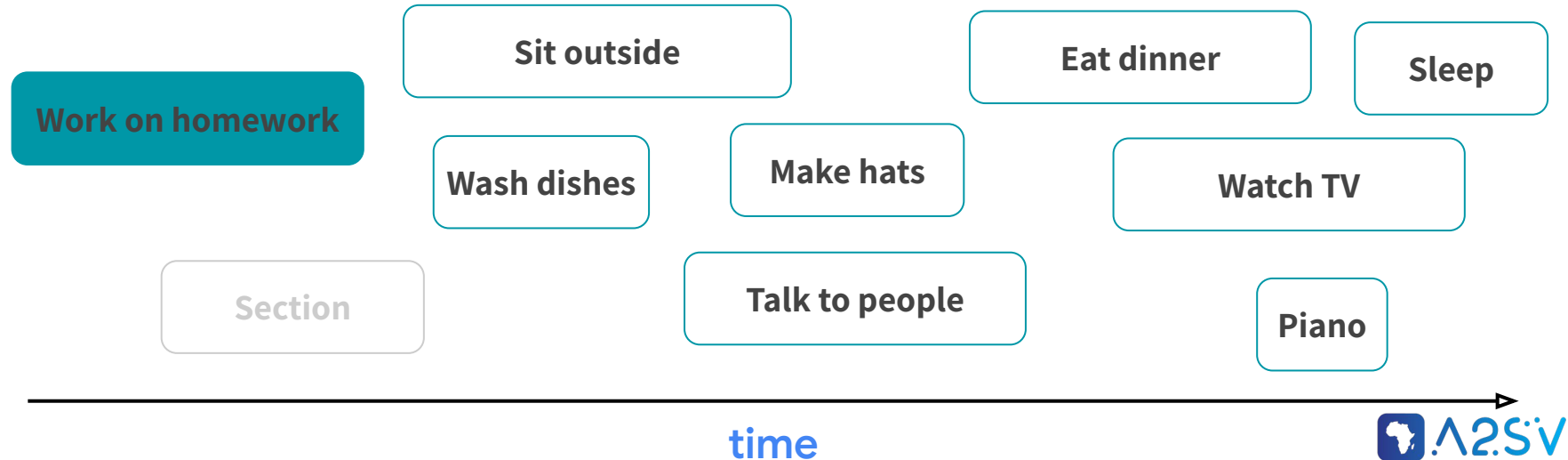
Our Greedy Algorithm

Pick an available activity with the smallest finish time & repeat.



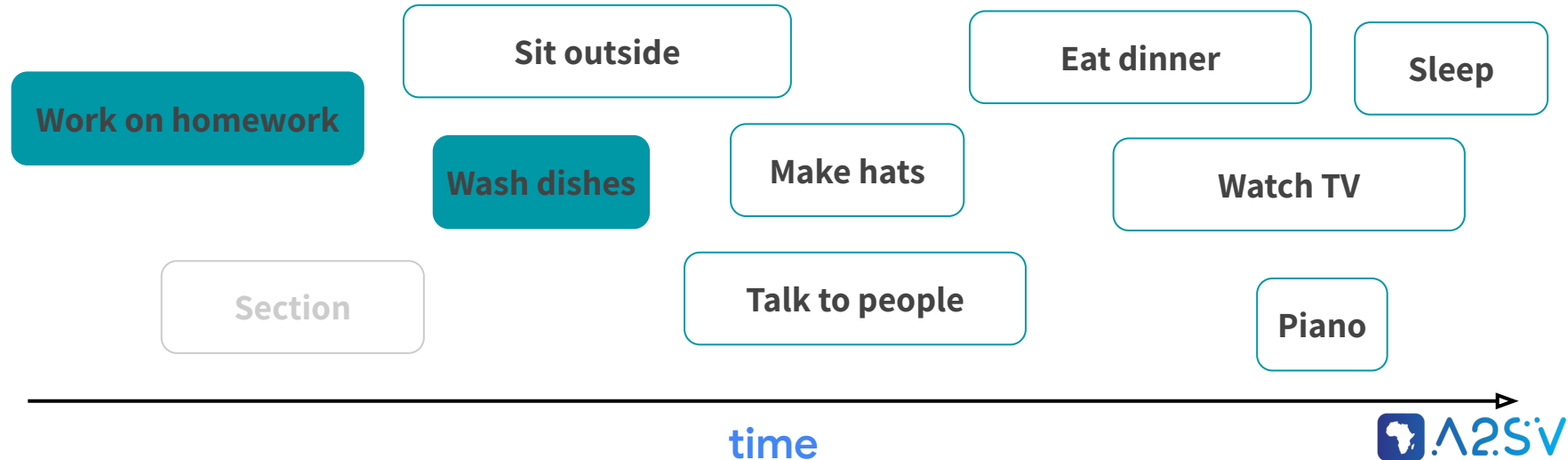
Our Greedy Algorithm

Pick an available activity with the smallest finish time & repeat.



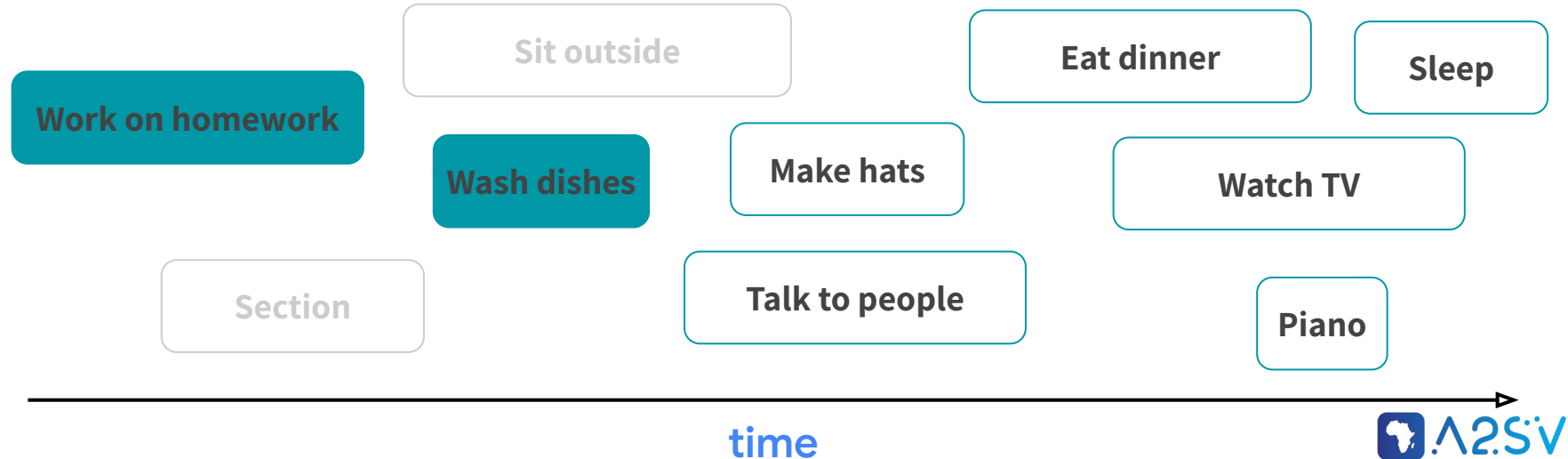
Our Greedy Algorithm

Pick an available activity with the smallest finish time & repeat.



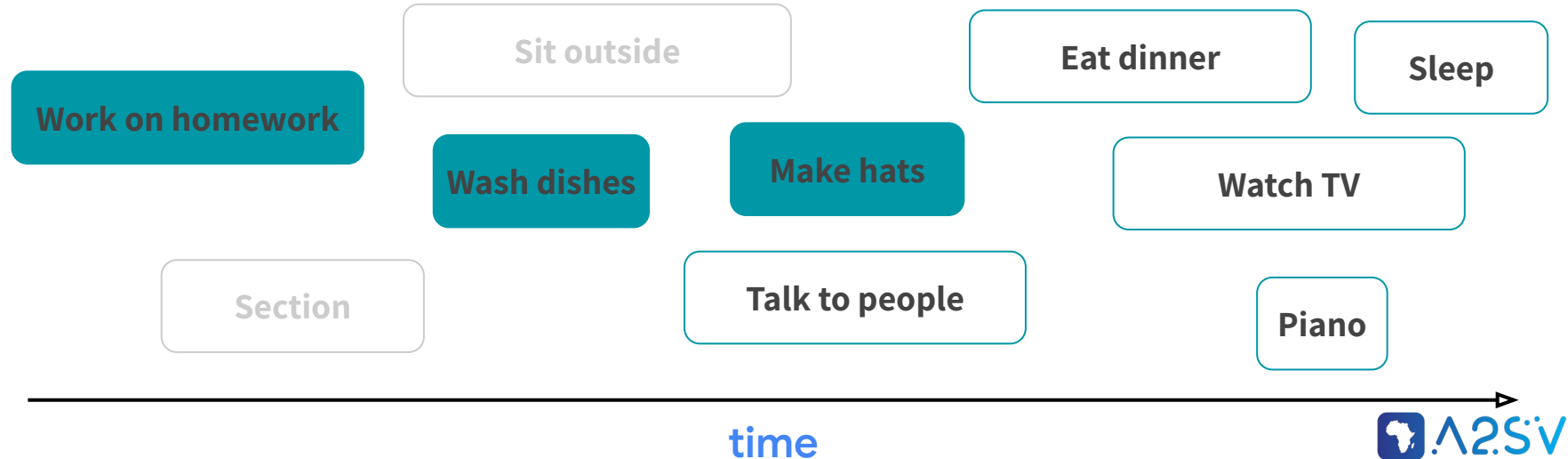
Our Greedy Algorithm

Pick an available activity with the smallest finish time & repeat.



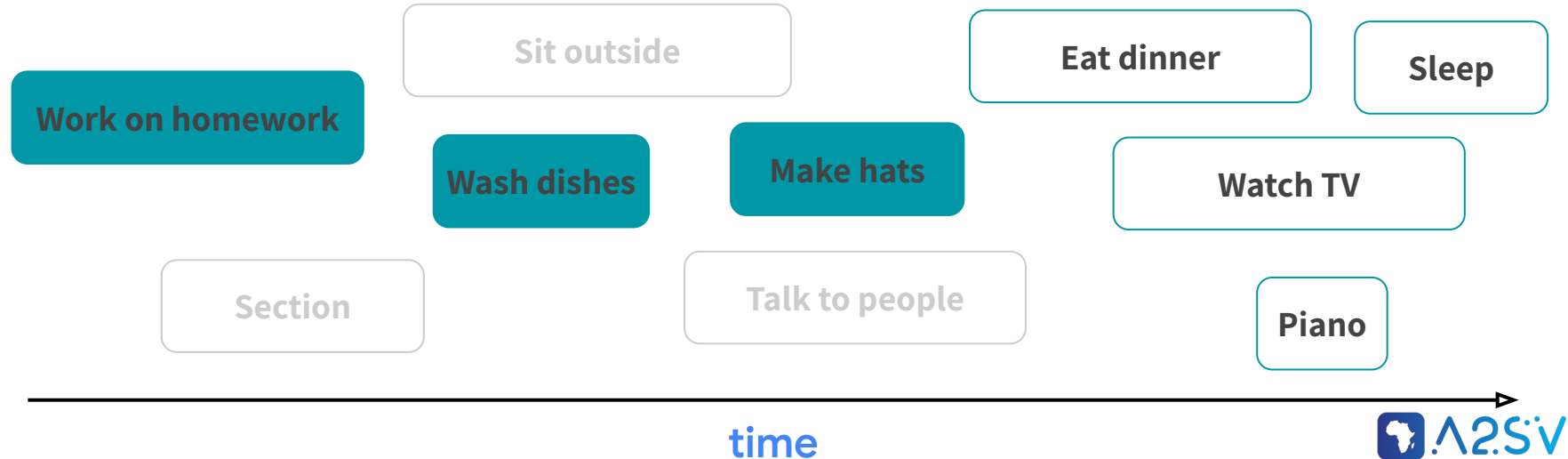
Our Greedy Algorithm

Pick an available activity with the smallest finish time & repeat.



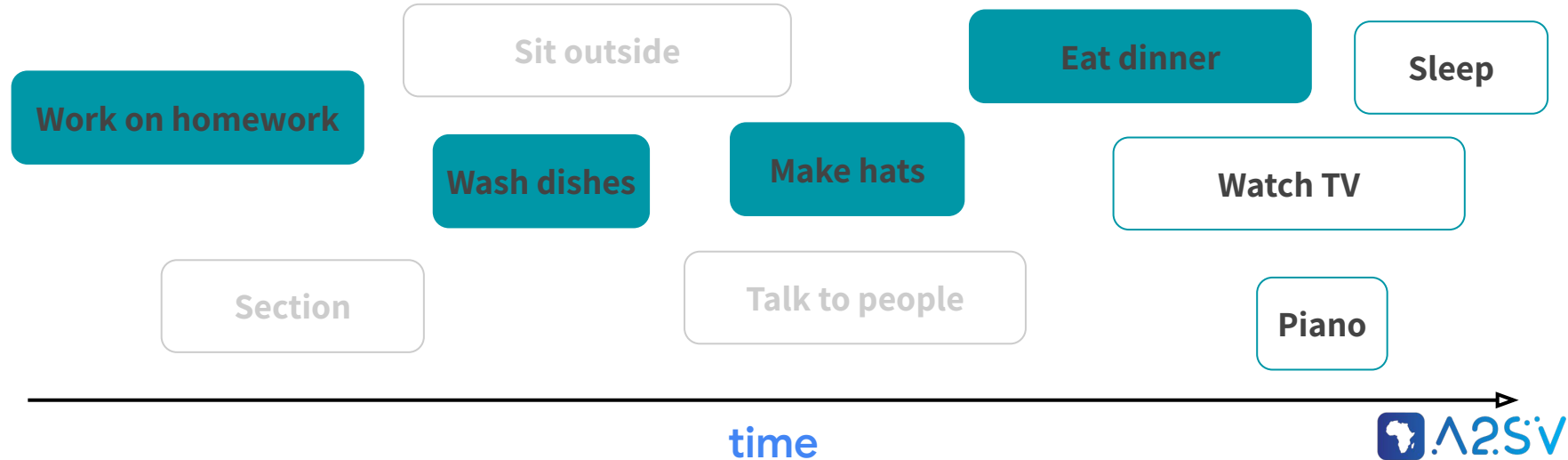
Our Greedy Algorithm

Pick an available activity with the smallest finish time & repeat.



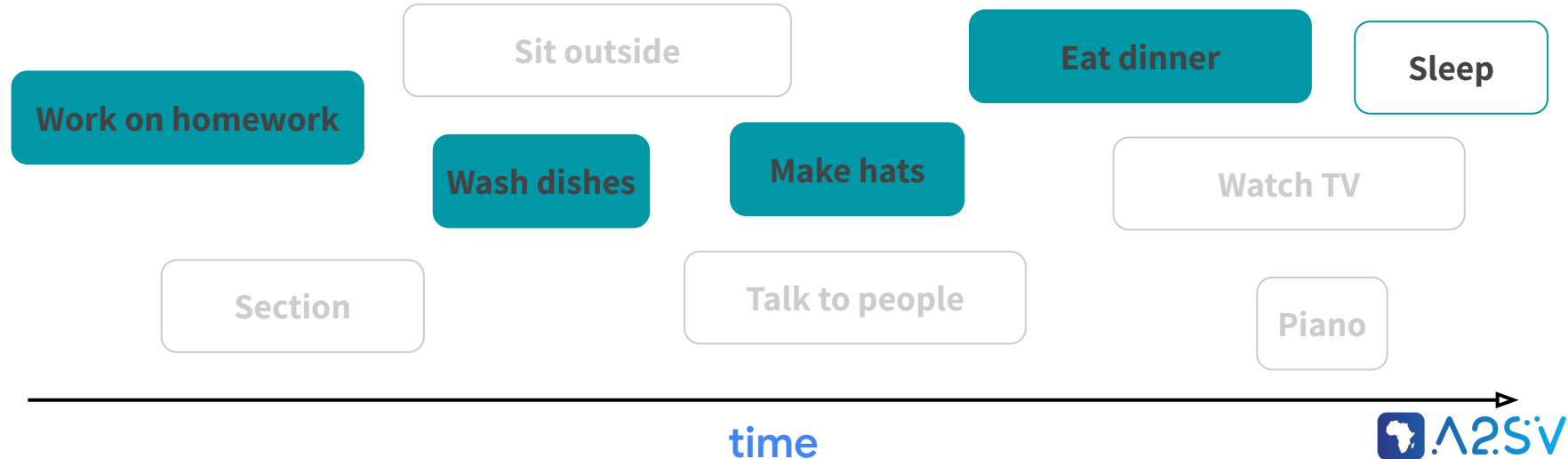
Our Greedy Algorithm

Pick an available activity with the smallest finish time & repeat.



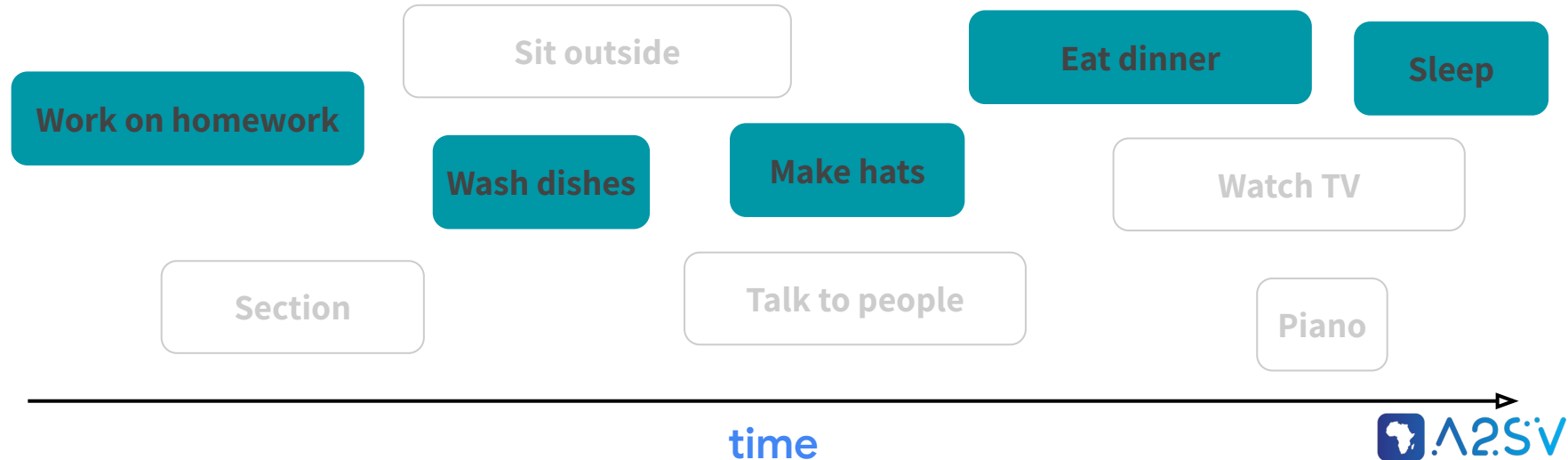
Our Greedy Algorithm

Pick an available activity with the smallest finish time & repeat.



Our Greedy Algorithm

Pick an available activity with the smallest finish time & repeat.



Why is it Greedy?

At each step in the algorithm, we make a choice (pick the available activity with the **smallest finish time**) and never look back.

How do we know that this greedy algorithm is correct?

(Proving correctness is **the hard part!**)

To prove correctness, we use induction.

In induction, we have:

- Base case
 - The initial case can be part of an optimal solution
- Inductive step
 - If the past k steps are part of the optimal solution and we make the $k+1$ choice greedily, we can still reach an optimal solution.

Activity Selection : Correctness

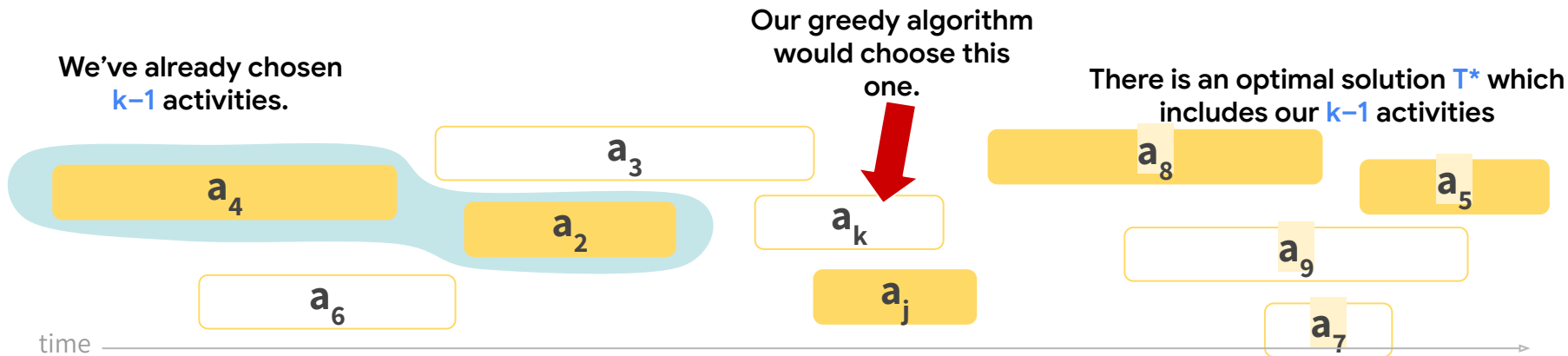
BASE CASE

There is an optimal solution that contains greedy activity 1 as first activity. Let A be an optimal solution with activity $k \neq 1$ as first activity. Then we can replace activity k (which has $F_k \geq F_1$) by activity 1. So, picking the first element in a greedy fashion works

INDUCTIVE STEP

Suppose we've already chosen $(k-1)$ activities, and we can reach an optimal solution from
After the first choice is made, remove all activities that are incompatible with the first chosen activity and recursively define a new problem consisting of the remaining activities. The first activity for this reduced problem can be made in a greedy fashion by the base principle.

To prove our induction, we can consider the case where there's an optimal solution that doesn't include our greedy choice.



Our greedy choice of a_k doesn't belong in T^* .

Then, let a_j be the activity in T^* after a_{k-1} .

Since a_k was chosen because it has the smallest end time, we know it ends before a_j . Thus, a_k doesn't conflict with anything in T^* after a_j . Swapping a_k with a_j would preserve optimality!

Pitfalls and Weaknesses

1. Greedy-Choice Property does not always lead to the Global Optimum
(Our intuition is wrong)
2. Failure to Consider Future Consequences
3. Dependency on Sorting
4. Proof Challenges (The biggest weakness)

Practice Problems

[Lemonade Change](#)

[Minimum Number of Arrows to Burst Balloons](#)

[Rabbits in Forest](#)

[Minimum Moves to Reach Target Score](#)

[Minimum Replacements to Sort the Array](#)

[Patching Array](#)

Quote of the Day

"A problem well stated is a problem half solved."

— John Dewey

DREAM
BIG.